

EXPERIMENT -5

Student Name: Vinay Tiwari

Branch: AIT-CSE (AIML)

Semester: 5

Subject Name: Database Management System

UID: 24BAI70919

Section/Group: 24AIT_KRG-1A

Subject Code: 24CSH-298

Q1) A fruit distribution company records the number of apples and oranges sold each day. For every date, exactly one record exists for apples and one record exists for oranges.

Generate a daily sales comparison report showing the difference between the number of apples sold and the number of oranges sold on the same day. The difference should be calculated as:

Difference = Apples Sold - Oranges Sold

Display the sale date and the calculated difference. The result should be sorted by sale date in ascending order.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

pgAdmin (for PostgreSQL)

EXPERIMENT -5

CODE SCREENSHOTS:

```
select sale_date, sum(case
when fruit = 'apples'
then sold_num else -sold_num end) from sales
group by sale_date
order by sale_date
```

OUTPUT SCREENSHOTS:

	sale_date date	sum bigint
1	2020-05-01	2
2	2020-05-02	0
3	2020-05-03	20
4	2020-05-04	-1

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand how to compare **daily apple and orange sales**.
- Learn to calculate the difference using the formula **Apples Sold – Oranges Sold**.
- Apply **data joining/grouping by sale date** to compare both products on the same day.
- Sort the report by **sale date in ascending order**.
- Present the calculated difference clearly for each date.

CONCLUSION:

In conclusion, this task demonstrates how to compare the daily sales of apples and oranges and determine the difference between them. By matching the sales records for each date, applying the given calculation, and sorting the results chronologically, we can create a clear and useful daily sales comparison report.

EXPERIMENT -5

Q2) A company stores hierarchical employee reporting information in a table. Each record represents a node in a hierarchy where:

- **id** represents the current node.
- **p_id** represents the parent node.

Classify every node into one of the following categories:

- **Root:** A node that does not have any parent.
- **Leaf:** A node that has a parent but does not have any children.
- **Inner:** A node that has both a parent and at least one child.

Display the node ID along with its category. Return the result sorted by node ID.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

pgAdmin (for PostgreSQL)

EXPERIMENT -5

CODE SCREENSHOTS:

```
SELECT
  t.id,
  CASE
    WHEN t.p_id IS NULL THEN 'Root'
    WHEN EXISTS (
      SELECT 1
      FROM Tree c
      WHERE c.p_id = t.id
    ) THEN 'Inner'
    ELSE 'Leaf'
  END AS category
FROM Tree t
ORDER BY t.id;
```

OUTPUT SCREENSHOTS:

	id [PK] integer	category text
1	1	Root
2	2	Inner
3	3	Leaf
4	4	Leaf
5	5	Leaf

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand the concept of a **hierarchical tree structure** using parent-child relationships.

EXPERIMENT -5

- Learn to identify **Root, Leaf, and Inner nodes** based on their relationships.
- Apply **conditional logic** to classify each node correctly.
- Use **self-joins or subqueries** to determine whether a node has a parent or children.
- Sort the final results by **node ID** for organized presentation.

CONCLUSION:

In conclusion, this task demonstrates how hierarchical data can be analysed and classified based on parent-child relationships. By checking whether each node has a parent and whether it has any children, we can accurately categorize nodes as Root, Leaf, or Inner. Sorting the results by node ID provides a clear and structured representation of the hierarchy.